SIMATS ENGINEERING

ASSESSMENT TOOL 2 – VIVA VOCE

COURSE CODE: CSA14 | COURSE NAME: COMPILER DESIGN

CO5 – Viva Voce Questions and Answers

Registration Number: 192524251

Q1. What is code optimization?

Code optimization is the process of improving intermediate or target code so that it executes faster, uses fewer resources, and produces the same correct output.

Q2. Why is optimization important in a compiler?

Optimization improves program performance, reduces execution time and memory usage, and can produce more efficient target code without changing program behavior.

Q3. What are the goals of code optimization?

The main goals are to reduce execution time, reduce memory usage, decrease code size, improve resource utilization, and preserve program correctness.

Q4. What are the principal sources of optimization?

Major sources include redundant computations, constant expressions, unreachable or dead code, inefficient loops, unnecessary loads and stores, and inefficient instruction sequences.

Q5. Give an example of redundant computation.

For example, if a program calculates x = a + b and later calculates y = a + b again without changes to a or b, the second calculation is redundant.

Q6. What is constant folding?

Constant folding evaluates an expression containing constants at compile time. For example, x = 5 * 4 can be changed to x = 20.

Q7. What is constant propagation?

Constant propagation replaces a variable with its known constant value. For example, if a = 10, then b = a + 5 can be optimized to b = 15.

Q8. What is strength reduction?

Strength reduction replaces an expensive operation with a cheaper equivalent operation, such as replacing multiplication by 2 with addition or a suitable shift operation.

Q9. What is algebraic simplification?

It simplifies expressions using algebraic identities. For example, x + 0 becomes x and x * 1 becomes x.

Q10. What is loop optimization?

Loop optimization improves the performance of loops by reducing unnecessary calculations or instructions, such as moving loop-invariant computations outside the loop.

Q11. What is peephole optimization?

Peephole optimization examines a small sequence of consecutive instructions and replaces it with an equivalent but more efficient sequence.

Q12. Why is it called 'peephole'?

It is called peephole optimization because the compiler looks through a small 'peephole' or window at a limited number of instructions at a time.

Q13. What are the types of peephole optimizations?

Common types include null sequence elimination, instruction replacement, algebraic simplification, redundant load/store elimination, constant folding, and strength reduction.

Q14. How does peephole optimization remove redundant instructions?

It identifies instructions whose results are unnecessary or duplicated and removes or replaces them with a shorter equivalent instruction sequence.

Q15. What is null sequence elimination?

Null sequence elimination removes instructions that have no useful effect, such as an instruction that produces a value which is never used.

Q16. What is instruction replacement?

Instruction replacement substitutes an inefficient instruction sequence with a more efficient instruction that produces the same result.

Q17. Give an example of algebraic simplification in peephole optimization.

An expression such as x + 0 can be replaced by x, eliminating an unnecessary addition instruction.

Q18. What is redundant load/store elimination?

It removes unnecessary memory load or store instructions when the required value is already available or a stored value is never needed.

Q19. What is constant folding in peephole optimization?

It evaluates constant operations in a small instruction sequence at compile time and replaces them with the resulting constant.

Q20. What is strength reduction in peephole optimization?

It replaces a relatively expensive operation with a cheaper equivalent operation within the instruction sequence.

Q21. What is a basic block?

A basic block is a straight-line sequence of instructions with one entry point and one exit point, with no branches into or out of the block except at its boundaries.

Q22. What are the properties of a basic block?

It has a single entry point, a single exit point, and instructions execute sequentially without internal control-flow branches.

Q23. What is a leader statement?

A leader is the first statement of a basic block. It is used as the starting point when dividing intermediate code into basic blocks.

Q24. What are the rules to identify leaders?

The first statement is a leader; any target of a conditional or unconditional jump is a leader; and any statement immediately following a jump is also a leader.

Q25. What is a control flow graph (CFG)?

A control flow graph is a directed graph representing the flow of control among basic blocks of a program.

Q26. What are nodes and edges in CFG?

Nodes represent basic blocks, while directed edges represent possible transfers of control from one basic block to another.

Q27. How are basic blocks connected in a CFG?

An edge is added from one block to another when execution can transfer from the first block to the second through normal sequencing or a branch/jump.

Q28. Why are basic blocks important for optimization?

They provide manageable regions for local optimization and form the nodes of the control flow graph used for global analysis.

Q29. What is a Directed Acyclic Graph (DAG)?

A DAG is a directed graph with no cycles. In compiler optimization, it represents computations and dependencies within a basic block.

Q30. Why is DAG used in basic block optimization?

A DAG exposes repeated computations and dependencies, helping identify common subexpressions, redundant computations, and opportunities for dead code elimination.

Q31. How is a DAG constructed?

Leaves are created for input variables or constants, and interior nodes represent operations. Equivalent operations can share the same node when their operands are the same.

Q32. What is a leaf node in DAG?

A leaf node represents an input value, variable, or constant used by expressions in the basic block.

Q33. What is an interior node in DAG?

An interior node represents an operation such as addition, subtraction, multiplication, or division and points to its operand nodes.

Q34. How does DAG help in common subexpression elimination?

If the same operation with the same operands already has a node in the DAG, the existing node can be reused instead of recomputing the expression.

Q35. How does DAG help in dead code elimination?

Values that are computed but never used in a required result can be identified and their unnecessary computations can be removed.

Q36. What is a label in DAG nodes?

A label identifies the variable names or values associated with a DAG node, showing which program values are represented by that computation.

Q37. What is the difference between DAG and syntax tree?

A syntax tree represents the hierarchical syntactic structure of an expression, while a DAG can share common subexpressions so that the same computation is represented only once.

Q38. What are the advantages of DAG representation?

It detects common subexpressions, reduces redundant computations, supports dead code elimination, and provides a compact representation of computations within a basic block.

Q39. What is live variable analysis?

Live variable analysis determines whether the current value of a variable may be used later. A variable is live if its value may be needed in the future.

Q40. What is available expression analysis?

It determines which expressions have already been computed and whose operands have not changed, so their existing results can be reused.

Q41. What is the difference between local and global optimization?

Local optimization operates within a single basic block, while global optimization considers multiple basic blocks and control-flow paths.

Q42. What is data dependency?

Data dependency exists when one instruction depends on a value produced or modified by another instruction, restricting the order in which they can safely execute.

Q43. What is a code generator?

A code generator is the compiler phase that converts intermediate representation into target-machine code while selecting efficient instructions and managing registers.

Q44. What are the issues in code generator design?

Important issues include instruction selection, register allocation, instruction ordering, addressing modes, target-machine constraints, and efficient use of available registers.

Q45. What is a target machine?

A target machine is the processor or machine architecture for which the compiler generates the final machine or assembly code.

Q46. What is instruction selection?

Instruction selection is the process of choosing target-machine instructions that efficiently implement operations in the intermediate representation.

Q47. What are addressing modes?

Addressing modes specify how an instruction obtains its operands, such as immediate, register, direct, indirect, or indexed addressing.

Q48. What is register allocation?

Register allocation assigns frequently used variables and temporary values to a limited number of processor registers to improve execution efficiency.

Q49. What is register spilling?

Register spilling occurs when there are more values requiring registers than available registers, so some values are temporarily stored in memory.

Q50. What is next-use information, and why is it important?

Next-use information indicates where a variable or temporary will be used next. It helps the code generator decide which values should remain in registers and which can be spilled.